

Dati del progetto

Mathota Arachchige Tharuth Deshan Perera
3B TLC
The Last Corridor
30/05/2026

Idea del gioco

Questo è un gioco da horror + survival. Dopo che inizi il gioco ti troverai in un menu dove puoi scegliere la modalità easy, medium e hard, che cambia la velocità degli zombie. Ti troverai in una stanza buia che è formata dai corridoi, muri e zombie. Tu devi sopravvivere e trovare una via per scappare. Per scappare ci sarà una porta bianca nel muro in un posto casuale. Tu devi cercare questa porta mentre gli zombie ti seguiranno per eliminarti. Tu avrai un'arma con i proiettili infiniti, puoi uccidere quanti zombie vuoi e per ogni eliminazione si aggiunge un punteggio allo tuo score. Quando entri dalla porta tu vincerai con uno score che di quanti zombie tu uccidi prima entrare dalla porta.

Questo non l'ho trovato su online. Da prima volevo creare un gioco da action o shooting. Mentre creavo questi giochi, mi cambiava idea di come il gioco dovrebbe essere (questo cambiava dalla situazione ero). Prima volevo fare un gioco dove un cavaliere uccide un drago per vincere (una mia idea, non l'ho visto su online). Ma non mi è venuto quel gioco a causa degli sprite che usavo. Come il secondo gioco che volevo fare era un gioco simile al super mario. Ma questo è un gioco molto comune e pensavo che tante persone faranno un gioco di questo tipo, perché non voglio fare un che farà qualcun altro, che voglio un mio gioco originale, con le mie idee. Alla fine ho deciso di fare un gioco che formato da un luogo di backroom, che sei intrappolato dentro e cerchi una via da scappare dagli zombie per fare questo anche horror, così il gioco non sarà noioso perché tu devi sempre correre cercando la porta per scappare. Non ho aggiunto gli indizi per trovare la porta perché pensavo che così il gioco sarà molto facile e noioso.

Analisi e progettazione

Io volevo prima creare il luogo dove tu apparirai, e poi volevo aggiungere gli zombie facendo questo gioco un po' horror. Ho messo la persona in fpp con una pistola, così si diverte un po' uccidendo gli zombie. Dopo ho capito che non è abbastanza questo e ho deciso di aggiungere una porta in un muro in un posto casuale, così ogni volta che inizia il gioco non troverai la porta nello stesso posto di prima, quindi devi cercare di nuovo la porta. Il mio obiettivo è che così il gioco non sarà alla persona che gioca, sapendo già che dove si trova la porta da scappare (ho messo una porta bianca, perché così si vede bene anche un po' da lontano, così non sarà fastidioso il gioco ai quelli che giocano). Ho aggiunto uno score così potete fare delle competenze per fare un score maggiore.

Ho usato pygame,sys,math e random.

All'avvio: mostra schermata iniziale che è il Menu, il giocatore può scegliere la difficoltà.

Quando il giocatore preme "Play":

- inizializza personaggio, nemici(zombie) e punteggio
- entra nel ciclo di gioco:
 - legge i tasti
 - aggiorna posizioni
 - controlla collisioni
 - aggiorna lo schermo

Se il giocatore muore- mostra schermata di Trapped in corridor

Se il giocatore muore- mostra schermata di Escaped the corridor

Ho usato Lista MAP_BASE e MAP per creare il labirinto e ho usato una matrice. MAP_BASE contiene la mappa, dove il numero 1 rappresenta i muri e lo 0 i corridoi vuoti. All'inizio di ogni partita, il gioco copia questi dati nella lista MAP, e si crea la mappa. Dentro MAP il codice ha il numero 3, che rappresenta la porta bianca per scappare e vincere il gioco.

Lista ZOMBIE SPAWNS contiene le coordinate. Così posso dare che dove gli zombie appaiono. Lista di dizionari ZOMBIES gestisce i nemici vivi, che contiene un dizionario per ogni zombie presente nel labirinto. Ho usato i dizionari perché così posso salvare diverse informazioni per ogni singolo mostro. Ho usato nel dizionario:

- "x" e "y": indicano la posizione dello zombie nella mappa in quel momento.
- "speed": indica la velocità dello zombie (che cambia in base alla difficoltà scelta nel menu).
- "hp": indica i punti vita (quanti colpi gli servono per morire).

Lista depth_buffer serve a far funzionare l'effetto 3D. Salva la distanza del muro più vicino per ogni fetta verticale dello schermo.

Lista active_zombies si svuota e si riempie continuamente mentre il giocatore si muove. Contiene solo i dati degli zombie. Il gioco li riordina in base alla distanza, per disegnare prima quelli lontani e poi quelli vicini.

Sviluppo e uso dell'AI

Io non volevo fare un gioco fatto dall'AI. Ho sempre deciso prima di scrivere che il gioco dovrebbe così (il mio obiettivo in quel momento). Ho detto all'intelligenza artificiale che io voglio questo e spiegami come si scrive, e lui me spiega e poi scrivo. Se sbaglio do chiedo di correggermi. Mi ha preso più tempo sicuramente per creare gli zombie e la persona con la pistola, perché è la parte più difficile. Ho usato gemini per farmi aiutare a fare il codice come fare e mi spiegava come si fanno alcune parti del codice. Ad esempio: Quando faccio mi da

un errore in rosso, io faccio uno screenshot di quello e lo do a gemini con il codice, così lui me lo corregge. Gemini diceva che dove devo scrivere il codice e come, e così ho scritto il codice. Onestamente la maggior parte del mio lavoro è stato fatto da gemini, perché non sono riuscito e non sapevo a scrivere il codice correttamente.

Test e risultati

Ogni volta che modifico qualcosa del gioco, ho fatto un test di quello. Non sono andato alla prossima modificazione finché confermo che la modificazione che ho fatto è corretto.

Test	Cosa ho fatto	Cosa mi aspettavo	Cosa è successo
1	Il personaggio tocca il nemico	Perde una quantità della vita	Il personaggio muore e il gioco finisce
2	Premo il tasto "F"	Diminuisce e ingrandisce lo schermo	Funziona
3	Premo il tasto "esc"	Si chiude il gioco	Funziona

Ho provato a correre il gioco e mi dava solo uno schermo nero. Non potevo cliccare niente, non potevo eliminare lo schermo premendo "X" e "esc". Se avevo più tempo, cambierei e avanzare di più il gioco. Adesso non trovo nessun bug.

Conclusioni e riflessione personale

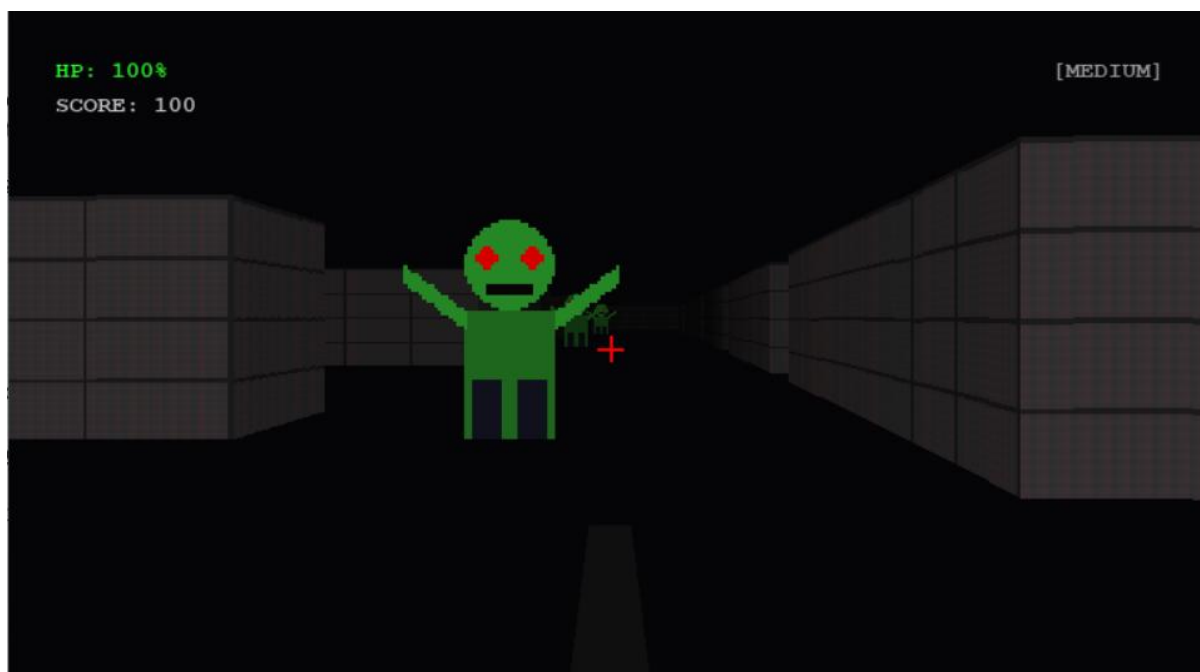
Ho capito un po' la base di pygame, ma mi manca ancora da sapere le cose come math, sys, random. Questo è molto interessante fare dei giochi ma anche è troppo difficile, come math, sys, random...che sono delle cose che non conosco ancora. Ma non sono ancora riuscito a capire bene, perché per me difficile capire coding dall'AI. Come che è la mia prima volta creare un gioco così è veramente faticoso fare da solo. Io prendo questo come una bella esperienza e continuare a capire le cose. Se rifaccio questo gioco, io modifico il grafico, cioè penso di mettere degli sprite con le animazioni realistiche. Voglio che

il gioco sarà realistico. Io non credo che ho usato l'AI in modo intelligente, perché ogni volta che non sono riuscito a fare qualcosa correttamente, lo dato a gemini da fare. Onestamente molte volte ho copiato e incollato il codice quando non sono riuscito a capire come funziona quella parte del codice, io dico che MI SONO APPOGGIATO MOLTO.

Questo è il menu del mio gioco. Si può scegliere easy,medium and hard e poi play.



Questo è il mio gioco nella partita



Lo schermo mio se vinco il gioco



Lo schermo se perdo il gioco

TRAPPED IN THE CORRIDOR

FINAL SCORE: 100

PRESS [SPACE] TO RETURN TO MENU

PRESS [ESC] TO QUIT

Posso premere “SPACE” per andare al menu, altrimenti “esc” per uscire dal gioco.

Il codice:

```
import pygame
import sys
import math
import random

pygame.init()
info = pygame.display.Info()
DESKTOP_W = info.current_w
DESKTOP_H = info.current_h

SCREEN_W = DESKTOP_W
SCREEN_H = DESKTOP_H
NUM_RAYS = 0
font = None
big_font = None
screen = None

SCALE = 2
FOV = math.pi / 3
HALF_FOV = FOV / 2
```



```

[1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,1],
[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1],
[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1],
[1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1]
]
MAP_W, MAP_H = len(MAP_BASE[0]), len(MAP_BASE)
MAP = []

# Posizioni fisse sulla mappa dove i nemici possono nascere
ZOMBIE_SPAWNS = [
    (300, 200), (700, 200), (1200, 200), (1800, 200),
    (400, 550), (900, 550), (1400, 550), (1900, 550),
    (500, 1100), (1100, 1100), (1600, 1100),
    (800, 1350), (1200, 1350), (1500, 900), (200, 1000)
]
ZOMBIES = []

TEX_SIZE = 64
# Disegno artificiale della texture del muro di mattoni
texture_wall = pygame.Surface((TEX_SIZE, TEX_SIZE))
texture_wall.fill((30, 30, 32))
for y in range(TEX_SIZE):
    for x in range(TEX_SIZE):
        if y % 16 != 0 and (x + (y//16)*32) % 32 != 0:
            texture_wall.set_at((x, y), (65 - (x%3)*6, 55 - (y%2)*4, 55))

# Disegno della texture della porta d'uscita bianca
texture_door = pygame.Surface((TEX_SIZE, TEX_SIZE))
texture_door.fill((230, 230, 235))
pygame.draw.rect(texture_door, (100, 100, 105), (4, 4, 56, 60), 3)

# Disegno dello sprite 2D dello zombie (faccia, occhi rossi, vestiti)
sprite_zombie = pygame.Surface((TEX_SIZE, TEX_SIZE), pygame.SRCALPHA)
pygame.draw.circle(sprite_zombie, (45, 160, 45), (32, 18), 12)
pygame.draw.circle(sprite_zombie, (255, 0, 0), (26, 16), 3.5)
pygame.draw.circle(sprite_zombie, (255, 0, 0), (38, 16), 3.5)
pygame.draw.rect(sprite_zombie, (10, 5, 5), (26, 23, 12, 3))
pygame.draw.rect(sprite_zombie, (35, 120, 35), (20, 30, 24, 34))
pygame.draw.rect(sprite_zombie, (20, 20, 35), (22, 48, 8, 16))
pygame.draw.rect(sprite_zombie, (20, 20, 35), (34, 48, 8, 16))
pygame.draw.line(sprite_zombie, (45, 160, 45), (20, 32), (4, 20), 5)
pygame.draw.line(sprite_zombie, (45, 160, 45), (44, 32), (60, 20), 5)

# Variabili di stato del giocatore
px, py = 150.0, 250.0
p_angle = 0.0
pitch = 0.0 # Gestisce lo sguardo verso l'alto o verso il basso
p_speed = 3.6

```



```

mouse_sensitivity = 0.0015
health = 100
score = 0
is_shooting = False
flash_timer = 0
damage_flash_timer = 0
target_zombie_idx = None
game_state = "MENU"
selected_difficulty = "medium"
zombie_damage = 0.65

# --- FUNZIONE DI RESET PARTITA (Logica del gioco) ---
def reset_game():
    global px, py, p_angle, pitch, health, score, MAP
    global ZOMBIES, game_state, zombie_damage

    # Crea una copia pulita della mappa per la nuova partita
    MAP = [row[:] for row in MAP_BASE]
    px, py = 150.0, 250.0
    p_angle = 0.0
    pitch = 0.0
    health = 100
    score = 0

    # GENERAZIONE CASUALE DELLA PORTA: Sostituisce un muro casuale (1) con la
    porta (3)
    while True:
        door_rx = random.randint(0, MAP_W - 1)
        door_ry = random.randint(0, MAP_H - 1)
        if MAP[door_ry][door_rx] == 1:
            dist_from_player = math.hypot(door_rx * TILE_SIZE + TILE_SIZE/2 - px, door_ry *
TILE_SIZE + TILE_SIZE/2 - py)
            # Evita che la porta appaia troppo vicina al giocatore all'inizio
            if dist_from_player > 500:
                MAP[door_ry][door_rx] = 3
                break

    # MODIFICATORI DI DIFFICOLTÀ: Cambiano i parametri di gioco basandosi sulla scelta
    del menu
    speed_mult = 1.0
    hp_bonus = 0

    if selected_difficulty == "easy":
        speed_mult = 0.6
        hp_bonus = -1
        zombie_damage = 0.2
    elif selected_difficulty == "medium":
        speed_mult = 1.0

```

```

    hp_bonus = 0
    zombie_damage = 0.65
elif selected_difficulty == "hard":
    speed_mult = 1.5
    hp_bonus = 1
    zombie_damage = 1.2

# Svuota e ricrea la lista degli zombie vivi applicando i modificatori
ZOMBIES.clear()
for zx, zy in ZOMBIE_SPAWNS:
    z_hp = max(1, random.randint(2, 3) + hp_bonus)
    ZOMBIES.append({
        "x": float(zx), "y": float(zy),
        "speed": random.uniform(0.8 * speed_mult, 1.8 * speed_mult),
        "hp": z_hp
    })

pygame.mouse.set_visible(False)
pygame.event.set_grab(True) # Blocca il mouse dentro la finestra di gioco
game_state = "PLAYING"

def draw_button(rect, text, is_active, is_hovered):
    color = (80, 80, 80)
    if is_active:
        color = (180, 40, 40)
    elif is_hovered:
        color = (120, 120, 120)

    pygame.draw.rect(screen, color, rect)
    pygame.draw.rect(screen, (255, 255, 255), rect, 3)

    lbl = font.render(text, True, (255, 255, 255))
    screen.blit(lbl, (rect.centerx - lbl.get_width()//2, rect.centery - lbl.get_height()//2))

#
=====
=====
# --- CICLO DI GIOCO PRINCIPALE (Game Loop) ---
#
=====
=====
running = True
while running:
    clock.tick(60)
    mouse_clicked = False

    for event in pygame.event.get():

```

```

    if event.type == pygame.QUIT or (event.type == pygame.KEYDOWN and event.key ==
pygame.K_ESCAPE):
        running = False

```

```

if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_f:
        is_fullscreen = not is_fullscreen
        set_window_mode(is_fullscreen)
    if event.key == pygame.K_SPACE and game_state in ["WIN", "GAMEOVER"]:
        game_state = "MENU"

```

```

# CONTROLLO DELLO SPARO (Click sinistro del mouse)

```

```

if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    if game_state == "PLAYING":
        if not is_shooting:
            is_shooting = True
            flash_timer = 4 # Durata del flash della pistola sullo schermo
            if target_zombie_idx is not None:
                ZOMBIES[target_zombie_idx]["hp"] -= 1 # Tolgo un punto vita al nemico

```

mirato

```

# Se lo zombie muore, aumenta il punteggio e rinasce lontano

```

```

if ZOMBIES[target_zombie_idx]["hp"] <= 0:
    score += 100
    while True:
        rx = random.randint(1, MAP_W - 2)
        ry = random.randint(1, MAP_H - 2)
        if MAP[ry][rx] == 0 and math.hypot(rx * 64 - px, ry * 64 - py) > 400:
            ZOMBIES[target_zombie_idx]["x"] = rx * TILE_SIZE + TILE_SIZE / 2
            ZOMBIES[target_zombie_idx]["y"] = ry * TILE_SIZE + TILE_SIZE / 2
            speed_mult = 0.6 if selected_difficulty == "easy" else (1.5 if
selected_difficulty == "hard" else 1.0)
            hp_bonus = -1 if selected_difficulty == "easy" else (1 if
selected_difficulty == "hard" else 0)
            ZOMBIES[target_zombie_idx]["speed"] = random.uniform(1.0 *
speed_mult, 2.2 * speed_mult)
            ZOMBIES[target_zombie_idx]["hp"] = max(1, random.randint(2, 3) +
hp_bonus)
            break

```

```

    elif game_state == "MENU":
        mouse_clicked = True

```

```

# --- SCHERMATA DEL MENU INIZIALE ---

```

```

if game_state == "MENU":
    pygame.mouse.set_visible(True)
    pygame.event.set_grab(False)
    screen.fill((15, 15, 20))

```

```

title_lbl = big_font.render("THE LAST CORRIDOR", True, (220, 50, 50))
screen.blit(title_lbl, (SCREEN_W//2 - title_lbl.get_width()//2, SCREEN_H * 0.2))

mouse_x, mouse_y = pygame.mouse.get_pos()

play_rect = pygame.Rect(SCREEN_W//2 - 150, SCREEN_H * 0.45, 300, 80)
easy_rect = pygame.Rect(SCREEN_W//2 - 250, SCREEN_H * 0.65, 150, 50)
med_rect = pygame.Rect(SCREEN_W//2 - 75, SCREEN_H * 0.65, 150, 50)
hard_rect = pygame.Rect(SCREEN_W//2 + 100, SCREEN_H * 0.65, 150, 50)

draw_button(play_rect, "PLAY", False, play_rect.collidepoint((mouse_x, mouse_y)))
draw_button(easy_rect, "EASY", selected_difficulty=="easy",
easy_rect.collidepoint((mouse_x, mouse_y)))
draw_button(med_rect, "MEDIUM", selected_difficulty=="medium",
med_rect.collidepoint((mouse_x, mouse_y)))
draw_button(hard_rect, "HARD", selected_difficulty=="hard",
hard_rect.collidepoint((mouse_x, mouse_y)))

if mouse_clicked:
    if play_rect.collidepoint((mouse_x, mouse_y)):
        reset_game()
    elif easy_rect.collidepoint((mouse_x, mouse_y)):
        selected_difficulty = "easy"
    elif med_rect.collidepoint((mouse_x, mouse_y)):
        selected_difficulty = "medium"
    elif hard_rect.collidepoint((mouse_x, mouse_y)):
        selected_difficulty = "hard"

# --- DURANTE IL GIOCO (Stato di gioco attivo) ---
elif game_state == "PLAYING":
    # Movimento dello sguardo tramite il movimento relativo del mouse
    mouse_dx, mouse_dy = pygame.mouse.get_rel()
    p_angle = (p_angle + mouse_dx * mouse_sensitivity) % (2 * math.pi)
    pitch = max(-SCREEN_H // 2, min(SCREEN_H // 2, pitch - mouse_dy * 1.2))

    # CONTROLLO MOVIMENTO GIOCATORE (WASD) E COLLISIONI CON I MURI
    keys = pygame.key.get_pressed()
    cos_a, sin_a = math.cos(p_angle), math.sin(p_angle)
    dx, dy = 0, 0
    if keys[pygame.K_w]: dx += cos_a; dy += sin_a
    if keys[pygame.K_s]: dx -= cos_a; dy -= sin_a
    if keys[pygame.K_a]: dx += sin_a; dy -= cos_a
    if keys[pygame.K_d]: dx -= sin_a; dy += cos_a

    if dx != 0 or dy != 0:
        length = math.hypot(dx, dy)
        nx, ny = px + (dx / length) * p_speed, py + (dy / length) * p_speed

```

```

        # Controlla separatamente l'asse X e l'asse Y sulla mappa di gioco per scivolare
lungo i muri
        map_val_x = MAP[int(py // TILE_SIZE)][int(nx // TILE_SIZE)]
        map_val_y = MAP[int(ny // TILE_SIZE)][int(px // TILE_SIZE)]

        if map_val_x == 0: px = nx
        elif map_val_x == 3: game_state = "WIN" # Se tocca il valore 3, ha trovato la porta e
vince
        if map_val_y == 0: py = ny
        elif map_val_y == 3: game_state = "WIN"

# COMPORTAMENTO DEGLI ZOMBIE (Inseguimento e Danno)
for z in ZOMBIES:
    zx, zy = z["x"], z["y"]
    zdx, zdy = px - zx, py - zy
    dist = math.hypot(zdx, zdy)

    # Se lo zombie è abbastanza vicino al giocatore, gli toglie la vita
    if dist < 32:
        health -= zombie_damage
        damage_flash_timer = 4 # Flash rosso di danno subito
        if health <= 0:
            game_state = "GAMEOVER" # Se la vita finisce, la partita è persa

    # Movimento dello zombie verso la posizione del giocatore
    if dist > 15:
        move_x = (zdx / dist) * z["speed"]
        move_y = (zdy / dist) * z["speed"]
        if MAP[int(zy // TILE_SIZE)][int((zx + move_x) // TILE_SIZE)] == 0: z["x"] +=
move_x
        if MAP[int((zy + move_y) // TILE_SIZE)][int(zx // TILE_SIZE)] == 0: z["y"] +=
move_y

    screen.fill((5, 5, 8))

#
=====
=====
# --- IL MOTORE GRAFICO 3D (RAYCASTING) ---
#
=====
=====
    depth_buffer = [float('inf')] * SCREEN_W # Inizializzazione dello Z-Buffer
monodimensionale
    start_angle = p_angle - HALF_FOV
    delta_angle = FOV / NUM_RAYS

```

```

# Ciclo per lanciare ogni singolo raggio visivo da sinistra a destra dello schermo
for ray in range(NUM_RAYS):
    ray_angle = start_angle + ray * delta_angle
    sin_r = math.sin(ray_angle) if math.sin(ray_angle) != 0 else 0.0001
    cos_r = math.cos(ray_angle) if math.cos(ray_angle) != 0 else 0.0001

    map_x, map_y = int(px // TILE_SIZE), int(py // TILE_SIZE)
    delta_dist_x = abs(1 / cos_r)
    delta_dist_y = abs(1 / sin_r)

    # Algoritmo DDA: calcola i passi da fare nella griglia per trovare un muro
    if cos_r < 0:
        step_x = -1
        side_dist_x = (px - map_x * TILE_SIZE) * delta_dist_x
    else:
        step_x = 1
        side_dist_x = ((map_x + 1) * TILE_SIZE - px) * delta_dist_x

    if sin_r < 0:
        step_y = -1
        side_dist_y = (py - map_y * TILE_SIZE) * delta_dist_y
    else:
        step_y = 1
        side_dist_y = ((map_y + 1) * TILE_SIZE - py) * delta_dist_y

    hit = False
    side = 0
    hit_val = 0
    steps = 0
    while not hit and steps < 100:
        steps += 1
        if side_dist_x < side_dist_y:
            side_dist_x += delta_dist_x * TILE_SIZE
            map_x += step_x
            side = 0
        else:
            side_dist_y += delta_dist_y * TILE_SIZE
            map_y += step_y
            side = 1
        if 0 <= map_x < MAP_W and 0 <= map_y < MAP_H:
            hit_val = MAP[map_y][map_x]
            if hit_val == 1 or hit_val == 3: hit = True # Il raggio si ferma se interseca un
muro o la porta
        else:
            break

    # Calcolo della distanza perpendicolare corretta del muro (evita l'effetto lente "occhio
di pesce")

```

```

if side == 0:
    perp_wall_dist = (side_dist_x - delta_dist_x * TILE_SIZE) / TILE_SIZE
    wall_x = py + perp_wall_dist * TILE_SIZE * sin_r
else:
    perp_wall_dist = (side_dist_y - delta_dist_y * TILE_SIZE) / TILE_SIZE
    wall_x = px + perp_wall_dist * TILE_SIZE * cos_r

perp_wall_dist *= math.cos(p_angle - ray_angle)
perp_wall_dist = max(0.1, perp_wall_dist)

# SALVATAGGIO NELLO Z-BUFFER: Salva la distanza calcolata per ogni colonna
di pixel dello schermo
for i in range(SCALE):
    idx = ray * SCALE + i
    if idx < SCREEN_W: depth_buffer[idx] = perp_wall_dist * TILE_SIZE

# Calcolo dell'altezza del muro da disegnare in base alla distanza (più è lontano, più
è piccolo)
wall_height = int(SCREEN_H * 1.2 / perp_wall_dist)
clipped_height = min(wall_height, SCREEN_H * 4)

# Calcolo della coordinata X sulla texture per proiettarla sul muro verticale
wall_x %= TILE_SIZE
tex_x = int(wall_x * (TEX_SIZE / TILE_SIZE))
if (side == 0 and cos_r > 0) or (side == 1 and sin_r < 0): tex_x = TEX_SIZE - tex_x -
1

# Decide se usare la texture del muro normale (1) o della porta (3)
active_texture = texture_door if hit_val == 3 else texture_wall

# Ritaglia una singola striscia verticale di texture e la scala all'altezza calcolata
sub_texture = pygame.Surface((1, TEX_SIZE))
sub_texture.blit(active_texture, (0, 0), (tex_x, 0, 1, TEX_SIZE))
scaled_strip = pygame.transform.scale(sub_texture, (SCALE, clipped_height))

# EFFETTO BUIO (Ombreggiatura): Scurisce i muri man mano che aumenta la
distanza dal giocatore
light_value = max(45, min(255, 255 - int(perp_wall_dist * 14)))
if side == 1: light_value = int(light_value * 0.7)

scaled_strip.fill((light_value, light_value, light_value),
special_flags=pygame.BLEND_RGBA_MULT)
screen.blit(scaled_strip, (ray * SCALE, (SCREEN_H // 2) - (clipped_height // 2) +
int(pitch)))

# PROIEZIONE E DISEGNO DEI NEMICI

```

```

target_zombie_idx = None
min_zombie_dist = float('inf')
mid_screen_x = SCREEN_W // 2

active_zombies = []
for idx, z in enumerate(ZOMBIES):
    sx = z["x"] - px
    sy = z["y"] - py
    dist = math.hypot(sx, sy)

    s_angle = math.atan2(sy, sx) - p_angle
    while s_angle > math.pi: s_angle -= 2 * math.pi
    while s_angle < -math.pi: s_angle += 2 * math.pi

    perp_sprite_dist = dist * math.cos(s_angle)
    if perp_sprite_dist < 1: perp_sprite_dist = 1

    # Filtra e seleziona solo gli zombie presenti all'interno del campo visivo (FOV)
    if perp_sprite_dist > 0 and abs(s_angle) < HALF_FOV + 0.4:
        active_zombies.append((perp_sprite_dist, s_angle, idx))

# ORDINAMENTO PER DISTANZA: Disegna prima i nemici lontani e poi quelli vicini
(Algoritmo del Pittore)
active_zombies.sort(key=lambda item: item[0], reverse=True)

for perp_sprite_dist, s_angle, idx in active_zombies:
    s_size = int(SCREEN_H * 1.2 / (perp_sprite_dist / TILE_SIZE))
    if s_size <= 0: continue

    s_screen_x = int((SCREEN_W / 2) + (math.tan(s_angle) * (SCREEN_W / 2) /
math.tan(HALF_FOV)))
    s_screen_y = (SCREEN_H // 2) - (s_size // 2) + int(pitch)

    start_x = s_screen_x - s_size // 2
    end_x = s_screen_x + s_size // 2

    # Disegna lo zombie colonna per colonna verticale, controllando la profondità dello
Z-Buffer
    for col in range(start_x, end_x):
        if 0 <= col < SCREEN_W:
            # CONTROLLO Z-BUFFER: Disegna il pixel dello zombie solo se è più vicino
del muro registrato
            if depth_buffer[col] > perp_sprite_dist:
                tex_col_x = int((col - start_x) * TEX_SIZE / s_size)
                if 0 <= tex_col_x < TEX_SIZE:

                    # Se il mirino al centro dello schermo tocca lo zombie, diventa il bersaglio
corrente

```



```

        if col == mid_screen_x and perp_sprite_dist < min_zombie_dist:
            min_zombie_dist = perp_sprite_dist
            target_zombie_idx = idx

    strip = pygame.Surface((1, TEX_SIZE), pygame.SRCALPHA)
    strip.blit(sprite_zombie, (0, 0), (tex_col_x, 0, 1, TEX_SIZE))
    scaled_strip = pygame.transform.scale(strip, (1, s_size))

    # Applica l'effetto oscurità anche allo zombie in base alla sua distanza
    z_light = max(110, min(255, 255 - int((perp_sprite_dist / TILE_SIZE) * 12)))
    scaled_strip.fill((z_light, z_light, z_light, 255),
special_flags=pygame.BLEND_RGBA_MULT)

    screen.blit(scaled_strip, (col, s_screen_y))

# --- ARMA E INTERFACCIA UTENTE (UI) ---
mid_x, mid_y = SCREEN_W // 2, SCREEN_H // 2
gun_offset_y = int(SCREEN_H * 0.02) if is_shooting else 0

# Gestione grafica della fiammata dello sparo
if is_shooting and flash_timer > 0:
    pygame.draw.circle(screen, (240, 130, 10), (mid_x, SCREEN_H - int(SCREEN_H *
0.22) + int(pitch * 0.4)), int(SCREEN_W * 0.045))
    flash_timer -= 1
else:
    is_shooting = False

# Disegno geometrico della canna della pistola in prima persona
gun_base = int(SCREEN_W * 0.035)
gun_tip = int(SCREEN_W * 0.018)
gun_len = int(SCREEN_H * 0.22)

pygame.draw.polygon(screen, (15, 15, 16), [
    (mid_x - gun_base, SCREEN_H),
    (mid_x - gun_tip, SCREEN_H - gun_len + gun_offset_y + int(pitch * 0.4)),
    (mid_x + gun_tip, SCREEN_H - gun_len + gun_offset_y + int(pitch * 0.4)),
    (mid_x + gun_base, SCREEN_H)
])

# Se il giocatore subisce danni, blitta un rettangolo rosso trasparente a schermo intero
if damage_flash_timer > 0:
    hurt_surf = pygame.Surface((SCREEN_W, SCREEN_H))
    hurt_surf.fill((180, 0, 0))
    hurt_surf.set_alpha(90)
    screen.blit(hurt_surf, (0, 0))
    damage_flash_timer -= 1

# Cambia il colore del mirino (Rosso se punta un nemico, Grigio se a vuoto)

```

```

reticle_color = (255, 0, 0) if target_zombie_idx is not None else (100, 100, 100)
pygame.draw.line(screen, reticle_color, (mid_x - 8, mid_y), (mid_x + 8, mid_y), 2)
pygame.draw.line(screen, reticle_color, (mid_x, mid_y - 8), (mid_x, mid_y + 8), 2)

# Scrittura dei testi informativi a schermo (HP, Punteggio e Difficoltà)
health_color = (255, 40, 40) if health < 35 else (40, 255, 40)
health_lbl = font.render(f"HP: {int(health)}%", True, health_color)
score_lbl = font.render(f"SCORE: {score}", True, (200, 200, 200))
diff_lbl = font.render(f"[{selected_difficulty.upper()})", True, (150, 150, 150))

screen.blit(health_lbl, (30, 30))
screen.blit(score_lbl, (30, 30 + health_lbl.get_height() + 5))
screen.blit(diff_lbl, (SCREEN_W - diff_lbl.get_width() - 30, 30))

# --- SCHERMATE DI FINE PARTITA (Vittoria o Sconfitta) ---
elif game_state in ["WIN", "GAMEOVER"]:
    pygame.mouse.set_visible(True)
    pygame.event.set_grab(False)

    # Sfondo verde scuro se vinci, rosso scuro se muori
    screen.fill((10, 25, 10) if game_state == "WIN" else (15, 0, 0))

    title_text = "ESCAPED THE CORRIDOR" if game_state == "WIN" else "TRAPPED IN
THE CORRIDOR"
    title_color = (40, 255, 40) if game_state == "WIN" else (255, 0, 0)

    title_lbl = big_font.render(title_text, True, title_color)
    final_score_lbl = font.render(f"FINAL SCORE: {score}", True, (255, 255, 255))
    restart_lbl = font.render("PRESS [SPACE] TO RETURN TO MENU", True, (120, 120,
120))
    exit_lbl = font.render("PRESS [ESC] TO QUIT", True, (120, 120, 120))

    screen.blit(title_lbl, (SCREEN_W // 2 - title_lbl.get_width() // 2, SCREEN_H // 3))
    screen.blit(final_score_lbl, (SCREEN_W // 2 - final_score_lbl.get_width() // 2,
SCREEN_H // 2))
    screen.blit(restart_lbl, (SCREEN_W // 2 - restart_lbl.get_width() // 2, SCREEN_H // 2 +
60))
    screen.blit(exit_lbl, (SCREEN_W // 2 - exit_lbl.get_width() // 2, SCREEN_H // 2 + 100))

    pygame.display.flip()

pygame.quit()
sys.exit()

```